UNIVERSIDAD AUTÓNOMA DEL PERÚ · DIRECCIÓN DE PLANEAMIENTO Y DESARROLLO INSTITUCIONAL

# Manual técnico de instalación y operación

Sistema de Formulación Presupuestal y Operativa POA - DPDI · versión web

**Dirigido a: área de Tecnologías de la Información**

> **Propósito:** describir la arquitectura, la instalación temporal en una laptop de la DPDI, la instalación definitiva en un servidor de la universidad, la migración de datos entre ambas, la publicación segura con HTTPS, los respaldos, el monitoreo, las actualizaciones y la solución de problemas.
>
> **Versión del documento:** 1.0 · Noviembre 2026
>
> **Contacto funcional:** Dirección de Planeamiento y Desarrollo Institucional (DPDI)

Documento de uso interno. Los nombres de equipos, rutas, dominios y direcciones IP de este manual son ejemplos y deben reemplazarse por los definidos por TI.

# Contenido

1. Arquitectura y componentes
2. Requisitos
3. Fase 1: instalación temporal en la laptop de la DPDI
4. Fase 2: instalación en un servidor de la universidad
5. Publicación con HTTPS (proxy inverso)
6. Migración de la laptop al servidor
7. Variables de configuración
8. Respaldos y restauración
9. Seguridad
10. Monitoreo y registros
11. Administración de usuarios desde el servidor
12. Actualización de versiones
13. Solución de problemas
14. Anexos: estructura de archivos y comandos rápidos

# 1 Arquitectura y componentes

El sistema es una aplicación web Python (Flask) servida por **Waitress**, un servidor WSGI de producción multiplataforma. Los datos se guardan en una base **SQLite** en modo WAL dentro de una carpeta de datos. Los usuarios solo necesitan un navegador.




La misma lógica de negocio (cálculos, directrices, exportación Excel/PDF) es compartida con la versión de escritorio.

| Componente | Descripción |
|---|---|
| `servidor_web.py` | Arranca Waitress. Parámetros `--host`, `--port`, `--threads` o variables `POA_HOST`, `POA_PORT`, `POA_THREADS`. |
| `web/` | Aplicación Flask: rutas, plantillas HTML, estilos y JavaScript (sin dependencias externas en el navegador). |
| `controllers.py`, `exporter.py`, `database.py` | Reglas de negocio, exportación del libro oficial y PDF, esquema y migraciones de la base. |
| `iniciar_servidor_web.bat` | Instalador/arranque para Windows: crea el entorno virtual, instala dependencias e inicia el servidor. |
| `respaldo_web.py` | Respaldo consistente en caliente de la base, con rotación. |
| `administrar_usuarios.py` | Herramienta de consola para listar, desbloquear, restablecer claves o crear administradores. |
| `Dockerfile`, `docker-compose.yml` | Despliegue en contenedor (opcional). |
| `manuales/` | Manuales en PDF publicados en la página «Ayuda» del sistema. |

## Carpeta de datos ( `POA_DATA_DIR` )

| Archivo | Contenido |
|---|---|
| `poa_dpdi_web.sqlite3` | Base de datos (más `-wal` y `-shm` mientras el servidor está activo). |
| `secret_key.txt` | Clave de firma de las sesiones. Si se pierde, todos los usuarios deben volver a ingresar. |
| `credenciales_iniciales.txt` | Clave inicial de `admin` (solo en la primera ejecución). Bórrelo tras el primer ingreso. |

| | |
|---|---|
| `marca/` | Logo institucional cargado por la DPDI. |
| `respaldos/` | Destino por defecto de `respaldo_web.py`. |

# 2 Requisitos

| Elemento | Laptop temporal | Servidor institucional |
|---|---|---|
| **Sistema operativo** | Windows 10/11 de 64 bits | Windows Server 2019/2022, o Linux (Ubuntu 22.04+/RHEL 9), o Docker |
| **CPU / RAM** | 2 núcleos / 4 GB | 2–4 vCPU / 4–8 GB |
| **Disco** | 2 GB libres | 10 GB (datos + 60 respaldos diarios) |
| **Software** | Python 3.11 o superior (3.12 recomendado). Dependencias: `requirements-web.txt` (Flask, Waitress, pandas, openpyxl, reportlab). | |
| **Red** | Cable de red a la LAN; puerto TCP 8080 abierto en el firewall | Registro DNS interno; puerto 443 (HTTPS) en el proxy; 8080 solo local |
| **Internet** | Solo para instalar dependencias (pip). En operación no se requiere, salvo que la DPDI configure una tipografía de Google Fonts. | |

## Capacidad

SQLite en modo WAL con escrituras breves admite sin problema decenas de usuarios trabajando a la vez, que es el perfil del proceso de formulación (cada responsable trabaja sobre su unidad). Si en el futuro se prevén cientos de usuarios simultáneos o alta disponibilidad, se recomienda migrar la capa de datos a PostgreSQL.

> **Importante**
> La carpeta de datos debe estar en un **disco local** del servidor. No la ubique en una unidad de red (SMB/NFS): SQLite puede corromperse con bloqueos de archivo remotos.

# 3 Fase 1: instalación temporal en la laptop de la DPDI

Mientras se habilita el servidor, el sistema puede funcionar en una laptop de la DPDI conectada a la red de la universidad. Los usuarios ingresan con `http://NOMBRE-LAPTOP:8080`.

## 3.1 Instalar Python

1. Descargue Python 3.12 (64 bits) desde `python.org/downloads`.
2. En el instalador marque **«Add python.exe to PATH»** y pulse «Install Now».
3. Verifique en una ventana de comandos (`Win`+`R` → `cmd`):

```
python --version
```

## 3.2 Copiar el sistema e iniciarlo

1. Copie la carpeta `sistema_poa_dpdi` a `C:\SistemaPOA\sistema_poa_dpdi` (evite rutas dentro de OneDrive o Descargas).
2. Haga doble clic en `iniciar_servidor_web.bat`. La primera vez crea el entorno `.venv` e instala las dependencias (requiere internet, 2–5 minutos).
3. La ventana mostrará la dirección, por ejemplo `http://LAPTOP-DPDI:8080`. **No cierre esa ventana**: cerrarla detiene el sistema.
4. Abra `C:\SistemaPOA\sistema_poa_dpdi\datos_web\credenciales_iniciales.txt`, copie la clave de `admin`, ingrese en `http://localhost:8080`, cambie la clave y **borre el archivo**.
5. Entregue la cuenta de administrador a la DPDI (que creará los usuarios desde el sistema).

## 3.3 Permitir el acceso desde otros equipos (firewall)

En una ventana de comandos **como administrador**:

```
netsh advfirewall firewall add rule name="Sistema POA DPDI" dir=in action=allow
protocol=TCP localport=8080 profile=domain,private
```

Si Windows muestra la ventana «Firewall de Windows Defender bloqueó algunas características», permita el acceso en redes de dominio/privadas. No habilite el perfil «Público».

## 3.4 Nombre o IP de la laptop

```
hostname
ipconfig
```

Pruebe desde otro equipo `http://NOMBRE-LAPTOP:8080`. Si el nombre no resuelve, use la IPv4 (`http://10.x.x.x:8080`). Se recomienda que TI **reserve la IP** de la laptop en el DHCP (o cree un alias DNS, por ejemplo `poa-temporal`) para que la dirección no cambie.

## 3.5 Evitar suspensión y apagados

```
powercfg /change standby-timeout-ac 0
powercfg /change hibernate-timeout-ac 0
powercfg /change monitor-timeout-ac 15
```

- Mantenga la laptop **conectada a la corriente y por cable de red**. Configure «Al cerrar la tapa: no hacer nada» (Panel de control → Opciones de energía).
- Programe las actualizaciones de Windows fuera del horario de formulación (Horas activas).
- Algunas redes Wi-Fi aíslan a los clientes entre sí; si los usuarios no llegan a la laptop por Wi-Fi, use cable.

## 3.6 Inicio automático (opcional)

Para que el sistema arranque al iniciar sesión en la laptop:

```
schtasks /Create /TN "Sistema POA DPDI" /SC ONLOGON /TR
"C:\SistemaPOA\sistema_poa_dpdi\iniciar_servidor_web.bat"
```

## 3.7 Respaldo diario en la laptop

```
schtasks /Create /TN "Respaldo POA DPDI" /SC DAILY /ST 19:00 /TR
"C:\SistemaPOA\sistema_poa_dpdi\.venv\Scripts\python.exe
C:\SistemaPOA\sistema_poa_dpdi\respaldo_web.py --db
C:\SistemaPOA\sistema_poa_dpdi\datos_web\poa_dpdi_web.sqlite3 --destino D:\RespaldosPOA
--conservar 30"
```

Copie periódicamente la carpeta de respaldos a un repositorio institucional (OneDrive/SharePoint de la DPDI o un servidor de archivos).

> **Limitaciones de la etapa temporal**
> La disponibilidad depende de que la laptop esté encendida y en red; no hay HTTPS (la comunicación viaja sin cifrar dentro de la LAN). Úsela solo en la red interna y por el menor tiempo posible.

# 4 Fase 2: instalación en un servidor de la universidad

Elija una de las tres opciones. En todas, la carpeta de datos debe quedar **fuera** de la carpeta del programa, para que las actualizaciones no la toquen (por ejemplo `D:\DatosPOA` o `/var/lib/poa`).

## Opción A — Windows Server como servicio (NSSM)

1. Instale Python 3.12 para todos los usuarios (marque «Add to PATH» e «Install for all users»).
2. Copie el programa a `C:\SistemaPOA\sistema_poa_dpdi` y cree el entorno:

```
cd C:\SistemaPOA\sistema_poa_dpdi
python -m venv .venv
.venv\Scripts\pip install -r requirements-web.txt
mkdir D:\DatosPOA\logs
```

1. Descargue NSSM (`nssm.cc`), copie `nssm.exe` a `C:\SistemaPOA\` y registre el servicio (consola como administrador):

```
set N=C:\SistemaPOA\nssm.exe
%N% install SistemaPOA "C:\SistemaPOA\sistema_poa_dpdi\.venv\Scripts\python.exe"
"C:\SistemaPOA\sistema_poa_dpdi\servidor_web.py"
%N% set SistemaPOA AppDirectory C:\SistemaPOA\sistema_poa_dpdi
%N% set SistemaPOA AppEnvironmentExtra POA_DATA_DIR=D:\DatosPOA POA_PORT=8080
PYTHONUNBUFFERED=1
%N% set SistemaPOA AppStdout D:\DatosPOA\logs\servicio.log
%N% set SistemaPOA AppStderr D:\DatosPOA\logs\servicio.log
%N% set SistemaPOA AppRotateFiles 1
%N% set SistemaPOA AppRotateBytes 10485760
%N% set SistemaPOA Start SERVICE_AUTO_START
%N% start SistemaPOA
```

1. Recomendado: ejecute el servicio con una cuenta de servicio sin privilegios (por ejemplo `svc_poa`) con permiso de **modificar** solo en `D:\DatosPOA` y de lectura en `C:\SistemaPOA`: `%N% set SistemaPOA ObjectName .\svc_poa CLAVE`.
2. Abra el puerto en el firewall solo si no usará proxy HTTPS (ver 3.3); si usa proxy, defina además `POA_HOST=127.0.0.1`.

> **Sin NSSM**
> Alternativa: Programador de tareas con desencadenador «Al iniciar el sistema», cuenta de servicio, acción `python.exe servidor_web.py`, carpeta de inicio la del programa y variables definidas a nivel de sistema (`setx /M POA_DATA_DIR D:\DatosPOA`). En «Configuración», marque reiniciar si la tarea falla.

## Opción B — Linux con systemd

```
sudo useradd --system --home /var/lib/poa --shell /usr/sbin/nologin poa
sudo mkdir -p /opt/sistema_poa /var/lib/poa && sudo chown poa:poa /var/lib/poa
sudo cp -r sistema_poa_dpdi /opt/sistema_poa/
cd /opt/sistema_poa/sistema_poa_dpdi
sudo python3.12 -m venv .venv && sudo .venv/bin/pip install -r requirements-web.txt
```

Archivo `/etc/systemd/system/sistema-poa.service`:

```
[Unit]
Description=Sistema POA - DPDI (web)
After=network.target

[Service]
User=poa
WorkingDirectory=/opt/sistema_poa/sistema_poa_dpdi
Environment=POA_DATA_DIR=/var/lib/poa
Environment=POA_HOST=127.0.0.1
Environment=POA_PORT=8080
Environment=POA_HTTPS=1
Environment=POA_PROXY=127.0.0.1
Environment=PYTHONUNBUFFERED=1
ExecStart=/opt/sistema_poa/sistema_poa_dpdi/.venv/bin/python servidor_web.py
Restart=on-failure
NoNewPrivileges=true

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now sistema-poa
sudo journalctl -u sistema-poa -f
```

## Opción C — Docker

```
cd sistema_poa_dpdi
mkdir -p datos && sudo chown 1000:1000 datos     # usuario «poa» del contenedor
docker compose up -d --build
docker compose logs -f
```

Los datos quedan en `./datos` (volumen `/datos`). El contenedor incluye un *healthcheck* contra `/salud` y se reinicia automáticamente (`restart: unless-stopped`). Para HTTPS, descomente `POA_HTTPS` y `POA_PROXY` en `docker-compose.yml` y publique el puerto solo en `127.0.0.1:8080:8080` si el proxy está en el mismo host.

## Primer arranque en el servidor

- Si es una instalación nueva, la clave de `admin` queda en `credenciales_iniciales.txt` de la carpeta de datos (o defina `POA_ADMIN_PASSWORD` antes del primer arranque). Bórrelo después del primer ingreso.
- Si viene de la laptop, siga el capítulo 6 **antes** del primer arranque.
- Verifique `http://localhost:8080/salud` → `{"ok": true, ...}`.

# 5 Publicación con HTTPS (proxy inverso)

En el servidor definitivo, publique el sistema con un nombre DNS y certificado institucional (por ejemplo `https://poa.autonoma.pe`, nombre referencial). Waitress queda escuchando solo en `127.0.0.1:8080` y el proxy atiende el puerto 443. Defina en el servicio: `POA_HOST=127.0.0.1`, `POA_HTTPS=1` (cookie segura) y `POA_PROXY=127.0.0.1` (para registrar en la bitácora la IP real del usuario).

## Nginx

```
server {
    listen 80;
    server_name poa.autonoma.pe;
    return 301 https://$host$request_uri;
}
server {
    listen 443 ssl http2;
    server_name poa.autonoma.pe;
    ssl_certificate     /etc/ssl/certs/poa.crt;
    ssl_certificate_key /etc/ssl/private/poa.key;
    client_max_body_size 25m;                  # cargas de Excel
    add_header Strict-Transport-Security "max-age=31536000" always;
    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 300s;               # exportaciones consolidadas
    }
}
```

## IIS (Windows Server)

1. Instale los módulos **URL Rewrite** y **Application Request Routing (ARR)**. En ARR → «Server Proxy Settings» marque **Enable proxy**.
2. Cree un sitio con enlace HTTPS (puerto 443) y el certificado institucional. En «URL Rewrite → View Server Variables» agregue `HTTP_X_FORWARDED_PROTO` a las variables permitidas.
3. Coloque este `web.config` en la carpeta raíz del sitio:

```
<configuration>
  <system.webServer>
    <rewrite>
      <rules>
        <rule name="SistemaPOA" stopProcessing="true">
          <match url="(.*)" />
          <serverVariables>
            <set name="HTTP_X_FORWARDED_PROTO" value="https" />
          </serverVariables>
          <action type="Rewrite" url="http://127.0.0.1:8080/{R:1}" />
        </rule>
      </rules>
    </rewrite>
    <security><requestFiltering><requestLimits maxAllowedContentLength="26214400" />
</requestFiltering></security>
  </system.webServer>
</configuration>
```

ARR agrega `X-Forwarded-For` automáticamente. Agregue además una regla de redirección HTTP → HTTPS en el sitio del puerto 80.

**Acceso desde fuera del campus**

Se recomienda publicar el sistema solo en la red interna y dar acceso externo mediante la VPN institucional. Si se publica en Internet, hágalo únicamente con HTTPS, detrás del WAF o firewall perimetral de la universidad.

# 6 Migración de la laptop al servidor

1. Acuerde con la DPDI una ventana de corte (por ejemplo, fuera del horario laboral) y avise a los usuarios.
2. En la laptop, **detenga el servidor** (cierre la ventana de `iniciar_servidor_web.bat` o `Ctrl`+`C`) y confirme que no quedan procesos `python.exe` del sistema.
3. Copie la carpeta completa `datos_web` (base de datos, `secret_key.txt`, `marca/`, `respaldos/`) a la carpeta de datos del servidor (`D:\DatosPOA`, `/var/lib/poa` o `./datos` en Docker). Si existen archivos `-wal` / `-shm` con el servidor detenido, cópielos junto con la base.
4. Como alternativa sin detener el servicio, la DPDI puede descargar el **Respaldo completo** (Parámetros → Reportes y respaldo) y TI lo copia como `poa_dpdi_web.sqlite3` en la carpeta de datos del servidor (el logo se vuelve a subir).
5. En Linux/Docker ajuste el propietario: `sudo chown -R poa:poa /var/lib/poa` (o `1000:1000 ./datos`).
6. Inicie el servicio en el servidor, verifique `/salud`, ingrese con un administrador y revise que el tablero muestre las unidades y los estados de ficha esperados.
7. Comunique la nueva dirección. Opcional: deje en la laptop una página o aviso con la nueva URL, o cree un alias DNS con el nombre anterior.
8. Retire la instalación temporal: elimine la tarea programada, la regla de firewall y la carpeta de datos de la laptop (conserve un respaldo cifrado en el repositorio institucional).

```
schtasks /Delete /TN "Sistema POA DPDI" /F
schtasks /Delete /TN "Respaldo POA DPDI" /F
netsh advfirewall firewall delete rule name="Sistema POA DPDI"
```

## Migrar datos de la versión de escritorio (.exe)

Si alguna unidad trabajó con la versión de escritorio, su base está en `%APPDATA%\Sistema_POA_DPDI\poa_dpdi.sqlite3`. Puede usarse como base inicial del servidor (copiándola como `poa_dpdi_web.sqlite3`): al arrancar, el sistema agrega las tablas de usuarios y estados sin perder información. Para incorporar fichas sueltas, es más simple que la DPDI importe el libro Excel exportado por la unidad.

# 7 Variables de configuración

| Variable | Valor por defecto | Descripción |
|---|---|---|
| `POA_DATA_DIR` | `./data` ( `datos_web` con el .bat) | Carpeta de datos. |
| `POA_DB_PATH` | DATA_DIR/ `poa_dpdi_web.sqlite3` | Ruta de la base, si se quiere fuera de la carpeta de datos. |
| `POA_HOST` | `0.0.0.0` | Interfaz de escucha. Use `127.0.0.1` detrás de un proxy. |
| `POA_PORT` | `8080` | Puerto TCP. |
| `POA_THREADS` | `8` | Hilos de Waitress (peticiones simultáneas). |
| `POA_HTTPS` | `0` | `1` marca la cookie de sesión como segura (solo HTTPS). |
| `POA_PROXY` | (vacío) | IP del proxy inverso de confianza; habilita `X-Forwarded-For/Proto`. |
| `POA_SESION_HORAS` | `8` | Duración máxima de la sesión. |
| `POA_ADMIN_PASSWORD` | (aleatoria) | Clave inicial de `admin` en una instalación nueva. |
| `POA_SECRET_KEY` | `secret_key.txt` | Clave de firma de sesiones (si se prefiere gestionarla como secreto). |

Valores fijos de la aplicación: bloqueo de 15 minutos tras 5 intentos fallidos; tamaño máximo de archivo cargado 25 MB; clave mínima de 8 caracteres con letras y números.

# 8 Respaldos y restauración

## Respaldo automático

`respaldo_web.py` usa la API de respaldo de SQLite: obtiene una copia consistente **aunque haya usuarios conectados** y conserva los últimos N archivos ( `poa_dpdi_AAAAMMDD_HHMMSS.sqlite3` ).

```
REM Windows Server - diario a las 21:00 con la cuenta SYSTEM
schtasks /Create /TN "Respaldo POA DPDI" /SC DAILY /ST 21:00 /RU SYSTEM /TR
"C:\SistemaPOA\sistema_poa_dpdi\.venv\Scripts\python.exe
C:\SistemaPOA\sistema_poa_dpdi\respaldo_web.py --db D:\DatosPOA\poa_dpdi_web.sqlite3 --
destino E:\RespaldosPOA --conservar 60"
```

```
# Linux - crontab del usuario poa (crontab -e -u poa)
0 21 * * * cd /opt/sistema_poa/sistema_poa_dpdi && .venv/bin/python respaldo_web.py --db
/var/lib/poa/poa_dpdi_web.sqlite3 --destino /var/backups/poa --conservar 60

# Docker
0 21 * * * cd /ruta/sistema_poa_dpdi && docker compose exec -T poa-web python
respaldo_web.py --conservar 60
```

- Incluya la carpeta de respaldos y `marca/` en la política de copias de la universidad (copia fuera del servidor).
- Durante la etapa de formulación considere dos respaldos diarios (13:00 y 21:00).
- No copie el archivo `.sqlite3` en uso con el Explorador o `cp` mientras el servicio está activo: use siempre `respaldo_web.py` .

## Restauración

1. Detenga el servicio ( `nssm stop SistemaPOA` , `systemctl stop sistema-poa` o `docker compose stop` ).
2. Renombre la base actual (por ejemplo `poa_dpdi_web.sqlite3.dañada` ) y elimine `poa_dpdi_web.sqlite3-wal` y `-shm` .
3. Copie el respaldo elegido como `poa_dpdi_web.sqlite3` en la carpeta de datos.
4. Inicie el servicio y verifique con un administrador. Informe a la DPDI la fecha y hora del respaldo restaurado (los cambios posteriores deben volver a registrarse).

Verificación de integridad de un archivo (opcional):

```
python -c "import sqlite3,sys; print(sqlite3.connect(sys.argv[1]).execute('PRAGMA
integrity_check').fetchone()[0])" RUTA\respaldo.sqlite3
```

# 9 Seguridad

## Controles incluidos en la aplicación

- Claves almacenadas solo como hash seguro (werkzeug: scrypt/PBKDF2); cambio obligatorio de la clave inicial o restablecida.
- Bloqueo temporal tras intentos fallidos; sesión con vencimiento; cookie `HttpOnly` y `SameSite`, segura con `POA_HTTPS=1`.
- Protección CSRF en todas las operaciones de escritura y verificación de permisos por rol y unidad en el servidor.
- Bitácora de auditoría con usuario, acción, detalle, fecha y dirección IP.
- Los archivos temporales de cargas y descargas se eliminan al terminar cada operación.

## Lista de verificación para TI

| ✓ | Control |
|---|---|
| ☐ | Publicación con HTTPS y certificado institucional ( `POA_HTTPS=1` , `POA_PROXY` , `POA_HOST=127.0.0.1` ). |
| ☐ | Puerto 8080 no accesible desde la red cuando hay proxy; acceso externo solo por VPN. |
| ☐ | Servicio ejecutado con cuenta sin privilegios; permisos de escritura solo en la carpeta de datos. |
| ☐ | Carpeta de datos y respaldos con acceso restringido a TI (contienen toda la formulación y los usuarios). |
| ☐ | `credenciales_iniciales.txt` eliminado tras el primer ingreso. |
| ☐ | Respaldos diarios programados y probados (al menos una restauración de prueba). |
| ☐ | Antivirus con exclusión de escaneo en tiempo real para `*.sqlite3-wal` de la carpeta de datos (evita bloqueos). |
| ☐ | Sistema operativo y Python con actualizaciones de seguridad; revisión periódica de `pip list --outdated` . |
| ☐ | Monitoreo de `/salud` integrado a la herramienta de TI. |

# 10 Monitoreo y registros

- **Disponibilidad:** `GET /salud` responde `{"ok": true, "hora": "..."}` sin autenticación. Úselo en Zabbix, PRTG, Nagios o similar.
- **Registro del servicio:** la salida de Waitress (inicio, errores) va a `D:\DatosPOA\logs\servicio.log` (NSSM), a `journalctl -u sistema-poa` (systemd) o a `docker compose logs`.
- **Auditoría funcional:** la bitácora del sistema (Administración → Bitácora) registra ingresos, intentos fallidos, cambios, cargas y descargas.
- **Espacio en disco:** vigile el crecimiento de la carpeta de respaldos.

# 11 Administración de usuarios desde el servidor

Para emergencias (por ejemplo, el único administrador olvidó su clave), ejecute en el servidor, con las mismas variables del servicio:

```
cd C:\SistemaPOA\sistema_poa_dpdi
set POA_DATA_DIR=D:\DatosPOA
.venv\Scripts\python administrar_usuarios.py listar
.venv\Scripts\python administrar_usuarios.py desbloquear ana.alva
.venv\Scripts\python administrar_usuarios.py restablecer ana.alva       # muestra una
clave temporal
.venv\Scripts\python administrar_usuarios.py crear-admin soporte.dpdi   # crea o
reactiva un administrador
```

```
# Linux
sudo -u poa POA_DATA_DIR=/var/lib/poa /opt/sistema_poa/sistema_poa_dpdi/.venv/bin/python
administrar_usuarios.py listar
# Docker
docker compose exec poa-web python administrar_usuarios.py restablecer admin
```

La clave temporal se muestra en pantalla y debe cambiarse en el primer ingreso. Las acciones de la consola no pasan por la bitácora web: regístrelas en el sistema de tickets de TI.

# 12 Actualización de versiones

1. Coordine con la DPDI una ventana de mantenimiento y ejecute un respaldo (`respaldo_web.py`).
2. Detenga el servicio.
3. Reemplace el contenido de la carpeta del programa por la nueva versión **sin tocar la carpeta de datos** (por eso debe estar fuera del programa). Conserve `.venv` o vuelva a crearlo.
4. Actualice dependencias: `.venv\Scripts\pip install -r requirements-web.txt` (Docker: `docker compose up -d --build`).
5. Inicie el servicio. Las **migraciones de la base son automáticas** al arrancar (agregan columnas o tablas nuevas sin borrar datos).
6. Verifique `/salud`, el ingreso y el tablero. Opcional: ejecute las pruebas automáticas con `python -m unittest discover -s tests`.

Si algo falla: detenga el servicio, restaure la versión anterior del programa y el respaldo del paso 1.

Para regenerar los manuales en PDF tras modificarlos: `node manuales/generar_pdf.js` (requiere Node.js y Playwright con Chromium).

# 13 Solución de problemas

| Síntoma | Causa probable | Solución |
|---|---|---|
| **«python no se reconoce como un comando»** | Python no está en el PATH. | Reinstale Python marcando «Add python.exe to PATH» o use `py -3.12`. |
| **Error al instalar dependencias** | Sin internet o proxy institucional. | `pip install --proxy http://proxy:puerto -r requirements-web.txt`, o instale desde un equipo con salida e importe la carpeta `.venv`. |
| **Funciona en la laptop pero no desde otros equipos** | Firewall, perfil de red Público o aislamiento Wi-Fi. | Regla del 3.3; perfil de red «Dominio/Privado»; usar cable; probar por IP. |
| **«OSError: [WinError 10048]» / «Address already in use»** | Otro proceso usa el puerto 8080 (o el servidor ya está abierto). | `netstat -ano \| findstr :8080`; cierre el proceso o use `POA_PORT=8081`. |
| **«database is locked»** | Carpeta de datos en red, antivirus o copia manual del archivo en uso. | Use disco local, exclusión de antivirus y `respaldo_web.py` para copias. |
| **Los usuarios pierden la sesión a cada rato** | Cookie segura sin HTTPS, o `secret_key.txt` cambió. | Con `POA_HTTPS=1` acceda solo por https; no borre `secret_key.txt`. |
| **La bitácora muestra la IP del proxy** | `POA_PROXY` no definido o IP distinta. | Defina `POA_PROXY` con la IP exacta desde la que el proxy se conecta. |
| **Error 413 al cargar un Excel** | Límite del proxy. | Nginx `client_max_body_size 25m`; IIS `maxAllowedContentLength`. |
| **Error 502/504 en el proxy** | Servicio detenido o exportación lenta. | Revise el servicio y el log; aumente `proxy_read_timeout`. |
| **Docker: «PermissionError: /datos»** | La carpeta `./datos` pertenece a root. | `sudo chown -R 1000:1000 ./datos`. |
| **Nadie puede ingresar como administrador** | Clave olvidada o cuenta bloqueada. | `administrar_usuarios.py crear-admin` o `restablecer` (capítulo 11). |
| **El logo no aparece** | No se cargó o formato no admitido. | La DPDI lo sube en Parámetros → Identidad gráfica (PNG, SVG, JPG, WEBP). |

# 14 Anexos: estructura de archivos y comandos rápidos

## Estructura del programa

```
sistema_poa_dpdi/
├── servidor_web.py            arranque del servidor web (Waitress)
├── iniciar_servidor_web.bat   instalación y arranque en Windows
├── respaldo_web.py            respaldo en caliente con rotación
├── administrar_usuarios.py    gestión de usuarios por consola
├── requirements.txt / requirements-web.txt
├── Dockerfile / docker-compose.yml
├── controllers.py  exporter.py  database.py  catalogs_seed.py
├── web/            app.py, schemas.py, templates/, static/
├── manuales/       pdf/ (publicados en «Ayuda»), fuente/, img/
├── tests/          pruebas automáticas
└── main_app.py, build_exe.bat   versión de escritorio (no se usa en el servidor)
```

## Comandos rápidos

| Tarea | Windows (NSSM) | Linux (systemd) |
|---|---|---|
| Iniciar / detener | `nssm start\|stop SistemaPOA` | `systemctl start\|stop sistema-poa` |
| Estado | `nssm status SistemaPOA` | `systemctl status sistema-poa` |
| Registro | `D:\DatosPOA\logs\servicio.log` | `journalctl -u sistema-poa -f` |
| Salud | `curl http://127.0.0.1:8080/salud` | |
| Respaldo manual | `python respaldo_web.py --db RUTA\poa_dpdi_web.sqlite3 --destino RUTA_RESPALDOS` | |

Sistema POA - DPDI · Manual técnico de instalación y operación v1.0 · Para uso del área de Tecnologías de la Información